Computación gráfica y ambientes virtuales (0494)

Práctico 03

Objetivo

El objetivo es **generar** las mallas de dos primitivas nuevas —un cilindro y un cono— a partir de pocos parámetros, en lugar de escribir sus vértices a mano, y ubicar varias piezas en una misma escena usando una **matriz de modelo** y un color diferente para cada una.

El trabajo requiere usar los tres módulos del Práctico 02. En el `struct Vertex` hay que cambiar los datos que agrupa —sale el color, entran la normal y las coordenadas de textura— por lo que hay que modificar el VAO y el *fragment shader*. No se trata de reescribir los módulos: cambió el contenido de la tupla, así que cambia la tupla y lo que se deriva de ella.

Punto de partida

Los módulos `Mesh`, `Shader` y `primitives` del Práctico 02, y el `ResourceManager` del Práctico 01. Se suma la biblioteca `glm` (*OpenGL Mathematics*), que aporta los tipos y las operaciones de álgebra lineal con la misma notación de GLSL. Es una biblioteca **sólo de encabezados**: alcanza con agregar su directorio a las rutas de inclusión.

Se mantiene una **caja negra**: la matriz de ajuste de vista y proyección, que se da escrita en la Parte 4. Es una porción de la constante del práctico anterior que no tiene respaldo teórico aún (Unidad VII); la otra parte de aquella constante —rotar y achicar el objeto— es justamente lo que a partir de ahora se arma con la matriz de modelo. El test de profundidad se mantiene habilitado como en el Práctico 02.

Parte 1 — La tupla del vértice

Un vértice se comparte entre dos caras **si y sólo si coinciden todos sus atributos**. Al cambiar los atributos el cubo conserva sus 24 vértices —la normal no es continúa en las aristas— mientras que la superficie lateral de un cilindro de 8 gajos pasa de 32 a 18 (explicado en clase).

```
// Vertex.h  --- punto de partida, no molde
struct Vertex {                             // Tupla de atributos para un vertice
    glm::vec3 position;                     // Posicion
    glm::vec3 normal;                       // Perpendicular a la superficie
    glm::vec2 tex_coords;                   // Coordenadas de textura (u, v)
};
```

Funcionalidades que debe tener.

- El VAO configura ahora **tres** atributos sobre el mismo buffer entrelazado, en las ubicaciones 0, 1 y 2.
- `cube()` devuelve la normal de cada cara. Sigue teniendo 24 vértices y 36 índices.
- El color deja de formar parte de los datos del vértice: pasa a ser un dato del objeto y llega al *fragment shader* por otro camino (Parte 3).
- La normal y las coordenadas de textura **se calculan y se guardan aunque todavía no se utilicen**. La iluminación y las texturas entran más adelante en la cursada.

Algunas cuestiones a pensar.

- Los desplazamientos de cada atributo dentro de la tupla, ¿se escribieron a mano en el Práctico 02 o se calculan? Ésta es la migración que responde aquella pregunta.
- ¿Cuántos lugares del código hubo que tocar para agregar un atributo? ¿Coincide con lo que se había previsto?
- Si un atributo se declara en el *shader* y no se usa, el compilador de GLSL puede eliminarlo. ¿Qué devuelve entonces una consulta por su ubicación, y por qué eso no es un error?

Parte 2 — El módulo `primitives` (generación paramétrica)

Un cilindro se describe por medio del radio, largo y cantidad de gajos; la malla debe regenerarse al cambiar cualquiera de estos valores: los vértices se calculan dentro de un bucle y ninguna coordenada se escribe a mano.

```
// Primitives.h  --- punto de partida, no molde
namespace primitives {

MeshData cube(float sx = 1.0f, float sy = 1.0f, float sz = 1.0f);

MeshData cylinder(float radio, float largo,
                  unsigned gajos, unsigned anillos = 1U);

MeshData cone(float radio, float conicidad,     // conicidad: angulo de apertura
              unsigned gajos, unsigned anillos = 1U);

}  // namespace primitives
```

Funcionalidades que debe tener.

- **Cada anillo lleva $N + 1$ vértices y no N.** En la costura la posición y la normal coinciden, pero la coordenada de textura vale 0 y 1 a la vez: un atributo difiere, así que el vértice hay que duplicarlo.
- El orden de los tres índices de cada triángulo respeta el criterio ya fijado en el Práctico 02 (antihorario mirando la cara desde afuera). **Dentro de un bucle ese orden deja de ser evidente**: el que sale natural al recorrer el cuadrilátero deja todos los triángulos invertidos.
- Las normales. En el lateral del cilindro la normal es **radial** y sale de normalizar la posición con la componente axial en cero (cuál coordenada es la que se anula depende de como se fije el sistema de referencia). En el lateral del cono, la normal, **no es radial**: está inclinada por el semiángulo de la conicidad.
- Si se modelan las tapas, las mismas tienen normal **axial** y por lo tanto nunca comparten vértices con el lateral.
- Ninguna función usa la API de OpenGL: son generadores de *datos*.

Algunas cuestiones a pensar.

- El cono queda parametrizado por el **ángulo de conicidad** y no por la altura. ¿Hace falta una función auxiliar que convierta una en otra?
- Los ángulos de la interfaz, ¿se reciben en grados o en radianes? Sea cual fuere la decisión, tiene que quedar documentada: es una fuente clásica de resultados que no se parecen a nada.
- En el **ápice** del cono la normal no está definida —dependería de qué gajo se mire—. Hay que tomar un criterio y dejarlo anotado. Con ese criterio, ¿el ápice es *un* vértice o son tantos como gajos?
- Si se quitaran las coordenadas de textura de la tupla, ¿cuántos vértices tendría el lateral de un cilindro de N gajos? ¿Cómo cambiaría el bucle?

Parte 3 — El módulo `Shader` (los `uniform`)

Un `uniform` es una variable del programa de *shaders* que se fija antes de ejecutar el *pipeline* y mantiene el mismo valor para todos los vértices y todos los fragmentos de esa llamada. Es el camino por el que

se asigna el color y la matriz de modelo. Hay que ampliar el módulo `Shader` con un método por cada *tipo* de `uniform` que haga falta.

```
// Shader.h  --- lo que se agrega. Punto de partida, no molde
void set_uniform(const std::string& nombre, const glm::mat4& m) const;
void set_uniform(const std::string& nombre, const glm::vec3& v) const;

// Segunda version: se consulta la ubicacion UNA vez y despues se usa el entero
int  loc(const std::string& nombre) const;
void set_uniform(int ubicacion, const glm::mat4& m) const;
```

Funcionalidades que debe tener.

- **No es una lista cerrada**: cada tipo nuevo de `uniform` que requiera el proyecto agrega una sobrecarga.
- Se usan las llamadas de acceso directo al estado (`glProgramUniform*`), coherentes con el estilo del resto de los módulos (DSA). Reciben el programa como argumento y por lo tanto **no exigen haberlo activado antes**.
- La consulta de la ubicación devuelve -1 si el `uniform` no existe o si el compilador lo eliminó por no usarse, y pasar -1 **no produce ningún error**: la llamada se ignora en silencio. Conviene que el módulo avise acerca del fallo al leer la ubicación.

Algunas cuestiones a pensar.

- La versión por nombre busca una cadena de texto dentro del programa enlazado, una vez por `uniform` y por cuadro, para obtener siempre el mismo número. ¿En qué momento conviene consultar la ubicación y guardarla?
- La ubicación de un `uniform`, ¿cambia alguna vez? ¿Bajo qué condición deja de ser válida la que se guardó?
- La mayoría de los tutoriales utilizan `glUniform*`, que cambia el `uniform` sobre el programa de shader *activo*. ¿Qué ocurre si se lo llama sin haber activado el programa y por qué esa falla es difícil de encontrar?

Parte 4 — El programa: una escena con varias piezas

Se dibujan **dos o tres primitivas** en la misma escena, cada una en un lugar distinto, con un tamaño distinto y un color distinto. Las mallas se generan una sola vez; lo que cambia entre una pieza y otra son los `uniform`.

```
// vertex shader
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoords;

out vec3 vNormal;

uniform mat4 uModel;      // la matriz del modelo ACTUAL
uniform mat4 uAjuste;     // caja negra hasta ver la Unidad VII

void main()
{
    vNormal     = aNormal;
    gl_Position = uAjuste * uModel * vec4(aPos, 1.0);
}
```

La matriz de ajuste se da escrita. Corrige la relación de aspecto de la ventana y niega el eje Z; las dos cosas son trabajo de la proyección:

```
const float ancho = 800.0f, alto = 600.0f;          // de la ventana

const glm::mat4 ajuste = glm::scale(glm::mat4(1.0f),
                                    glm::vec3(alto / ancho, 1.0f, -1.0f));
```

Recomendaciones.

1. Las funciones de `glm` **devuelven, no modifican**: `glm::translate(m, v);` no hace nada si no se asigna el resultado. Los ángulos van en radianes (`glm::radians()` es una función auxiliar para esto) y `glm::mat4(1.0f)` construye la identidad mientras que `glm::mat4()` **no** la garantiza.
2. El orden importa: en `model * v` se aplica primero la matriz que está más a la derecha. Como cada llamada a `glm` multiplica por derecha, **la última llamada que se escribe es la primera que se aplica**.
3. Conviene generar cada malla una sola vez, fuera del bucle de dibujo, y dejar dentro únicamente el cambio de `uniform` y la llamada de dibujado.
4. Que compile sin ningún *warning*.
5. Documentar mínimamente en cada interfaz las decisiones tomadas.

Cómo se sabe que está bien.

- Se ven dos o tres piezas, en distinto lugar, con distinto tamaño y distinto color, sin caras que aparezcan y desaparezcan.
- Cambiar la cantidad de gajos por parámetro y volver a correr: la malla se regenera sola, sin tocar una sola coordenada.
- **Modo de depuración de normales.** La normal no se usa para nada todavía, de modo que una normal mal calculada **no produce ningún problema**. Una forma de poder "ver" los valores es reemplazar una línea del *fragment shader* como se muestra a continuación:

```
// FragColor = vec4(uColor, 1.0);
   FragColor = vec4(vNormal * 0.5 + 0.5, 1.0);   // la normal, como color
```

  Cada dirección da un color distinto. El lateral del cilindro tiene que mostrar un degradé suave alrededor del eje de simetría de revolución y el cubo seis caras de color plano.
- **Verificación del orden de los índices sin dibujar nada.** Para cada triángulo (A, B, C), el producto vectorial $(B - A) \times (C - A)$ tiene que apuntar para el mismo lado que la normal de sus vértices. Son unas pocas líneas de código y permiten verificar por cálculo. El descarte de caras traseras está deshabilitado así que si el orden está invertido no se detecta.
- El programa puede informar por consola cuántos vértices y cuántos índices generó cada primitiva. Para el lateral de un cilindro de N gajos y dos anillos, tienen que ser $2\,(N + 1)$ vértices.

Opcional.

Agregar `sphere()` al módulo. Es el mismo método con un bucle más —anillos de latitud en lugar de dos anillos—; puede servir para modelar el cielo por ejemplo. No forma parte del trabajo.

Como en los prácticos anteriores, las interfaces mostradas son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno, y hay que poder explicarlas.